

* Unit 3 :- Inter Process Communication

-> It is a mechanism provided by the OS that allows processes to communicate with each other. It is known as IPC.

eg.

-> Processes executing concurrently in the OS may be either independent processes or co-operating processes.

-> In OS we have 2 or more processes that are executing at the same time in the OS. These processes may be — .

-> Independent Processes :-

-> They can't affect or be affected by the other processes executing in the system.

-> [Processes are running concurrently & they need to cooperate with each other. So all the tasks can run smoothly.]

* Co-operating processes - They can affect or be affected by the other processes executing in the system.

→ eg:- any process that shares with other process if 1 process is sharing data with another process then we can say for sure that process can affect the other process which it is sharing the data. So if 2 processes are sharing data then those kind of processes are known as Co-p.

*] Why we need IPC?

→ There are several reasons which allows processes to co-operate.

- 1] Information Sharing
- 2] Computation Speedup → divide into sub task
- 3] Modularity → diving the sys into several different modules.
- 4] Convenience

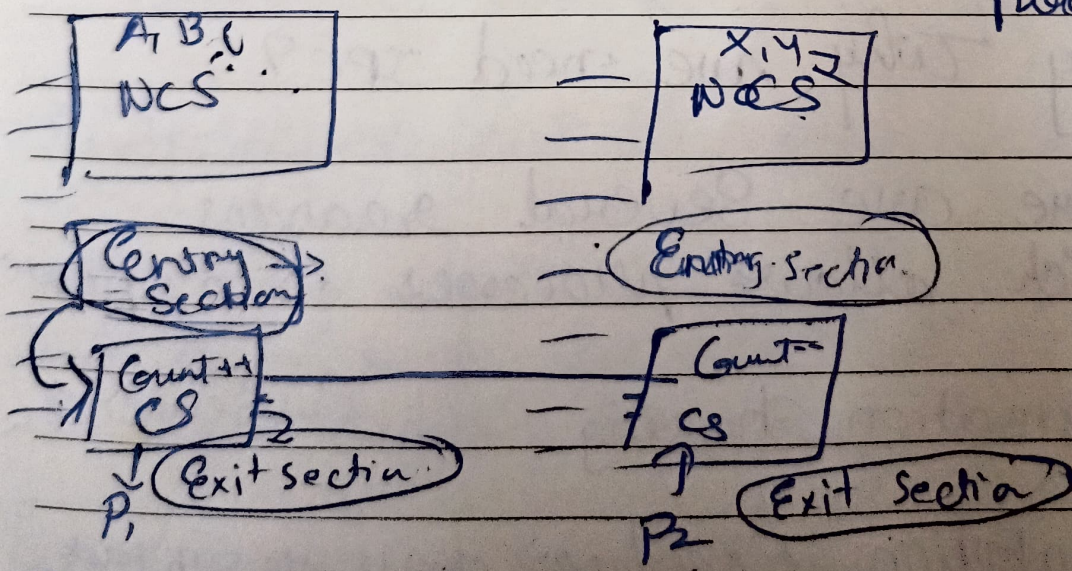
Problem

* Critical Section in OS:-

⇒ Process It is a code segment where processes access shared resources (like files, variables, printers) that need synchronized exclusive access to prevent data corruption (race condition).

→ It is part of the program where shared resources are accessed by various process.

P_1 P_2 → ~~the~~ placed where variables, resources are placed.



⊙ If both CS run together it create a problem it is known as Race Condition.

→ If we want make condition to not occur
→ we have to synchronize them.

→ Both program in CS & then
apply different methods..

→ for this ^{first}, we can write Entry section
code.

⊗ Race Condition:- It is a flaw
that occurs when two or more
processes or threads access &
manipulate a shared resource
concurrently, & the final outcome
depends on the unpredictable
order of their execution.

→ It is a situation that may
occur inside a critical
section.

→ It happens when the result of multiple thread execution in critical section differs according to the order in which threads execute.

④. Critical Section:-

→ when one process is executing in its critical section, no other process is allowed to execute in its critical section.

→ That is no, two processes are executing in their critical sections at the same time.

* Some Rules are there for the processes they are accessing the CS.

-> Each process must request permission to enter its CS.

→ The section of code implementing this request is the Entry Section.

→ The Critical Section may be followed by an Exit Section.

→ The remaining code is the Remainder Section.

do {

Control the entry into CS
→ gets a lock on required resources.

Entry Section ↑

Critical Section

Removes the lock
from the ← Exit Section
resources & let the other
known that its crs. is over.

Remainder Section
} while (TRUE);

* Solution:- To the Critical Section problem must satisfy the following three requirements

1. Mutual Exclusion:

→ If process A₁ is executing in its Critical Section, there no other processes can be executing in their CS.
If any other process is there
Be Here... Be Vibrant...
They must wait with process A₁.

11

2. Progress :- It means a process is not using the CS, then it should not stop any other process from accessing it.

→ Any process can enter a CS if it is free.

3. Bounded Waitings :-

It means that each process must have a limited waiting time. It should not wait endlessly to access the CS.

* Lea
Dirr'

Bounded Buffer Problem

* Producer & Consumer Problem

→ It is a standard problem of multi process synchronization.

→ In this we have 2 processes One for Producer & one is of Consumer

→ The assumption is both processes are coming at a same time. So both are parallel processes. & ^{co-operative} they are sharing something. (like Code, resources, memory or variable).

a common fixed sized buffer & use it as a queue.

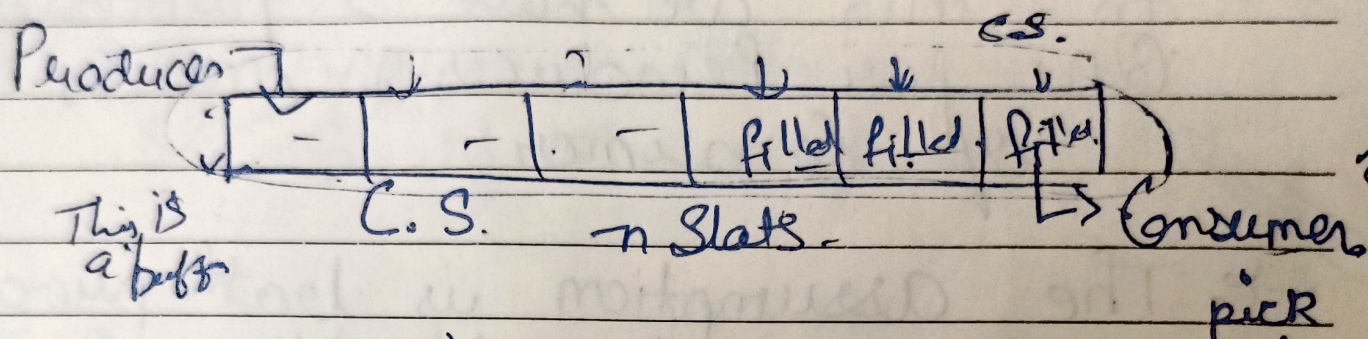
* Producer:- It generate data items & place them in a shared buffer.

* Consumer:- Remove & process data items from the buffer.

Problem..

- There is one thread of P & There is 1 thread of C

Producer produce data & Consumer consume that data.



* Problem:-(?) i want Synchronized method

C.S. → { Buffer } → Shared Resources.
 p & c should not insert & remove.
 → Syne b/w Pro & Con.

→ Pro. must not insert data when the buffer is full.

→ Consumer must not ~~stop~~ pick / remove data when the buffer is empty.

→ P tries to insert data into an empty slot of the buffer.
 C tries to remove data from a filled slot in the buffer.

* Bounded Buffer because block have finite buffer.

(*) Solution:-

① Producer:- Not produce any data when shared buffer is full.

② Consumer:- Not consume any data when shared buffer is empty.

③ The access shared buffer should be mutually exclusive.

(*) We solve this problem with the help of Semaphore.

→ It is like a special integer variable used for process synchronization.

→ It controls how many processes can access a shared resources.

* There are 3 types ^{of} problem ^{solve} using semaphores.

1. Full = 0 :- Counting Semaphore, number of filled slots. It prevents the ~~C~~ from trying to remove an item from an empty buffer.
2. Empty = N :- Counting Semaphore, number of empty slots in the buffer (Initially buffer size N). It prevents the ~~P~~ from adding an item to a full buffer.
3. Mutex = 1 :- Binary Semaphore for mutual exclusion.

mutex have decided which process should use the buffer.

It ensure that only one process modifies the buffer.

~~As we~~ They use 2 Signal.

→ A Particular process use the particular buffer. They use the buffer & produce data.

a. wait(): The process wants to use the resource (-1).

b. Signal(): The process has finished using the resource & release it (+1).

^{release} ^{from} Signal means release that resource

Producer Process.

function producer

① * Mutual Exclusion: Both P & C use mutex to ensure they don't access the buffer array or modify the in & out indices simultaneously. The mutex wait / signal pair brackets the C.S.

2. Buffer Bounds: - The empty Semaphore ensures the producer stops when the buffer is full (when empty reaches 0). The full Semaphore ensure the consumer stops when the buffer is empty (when full reaches 0).

3. Order

empty / full first, then mutex -> avoids deadlocks if a process blocks while holding the mutex

- 2. Access buffer inside mutex $\rightarrow$ ensure mutual exclusion.
- 1. Release mutex before signaling $\rightarrow$ buffer state is updated before waking other.

⑧ Reader's & Writer Problem

-2. A database is to be shared among several concurrent processes.

$\rightarrow$ Some processes may want only to read the database, where others may want to update (read & write) the database.

$\rightarrow$ Disting between these two types of processes by referring to the former as Readers & to the

- If Two Readers access the Shared data Simultaneously no adverse affects will result.
- If a writer & some other thread (a reader or a writer) access the database, Simultaneously, Chaos may ensue.
- To ensure that these difficulties don't arise, we required that the writers have exclusive access to the Shared database.
- This problem is referred as the Reader's-writers problem.

② Solution to RWP using Semaphores:-

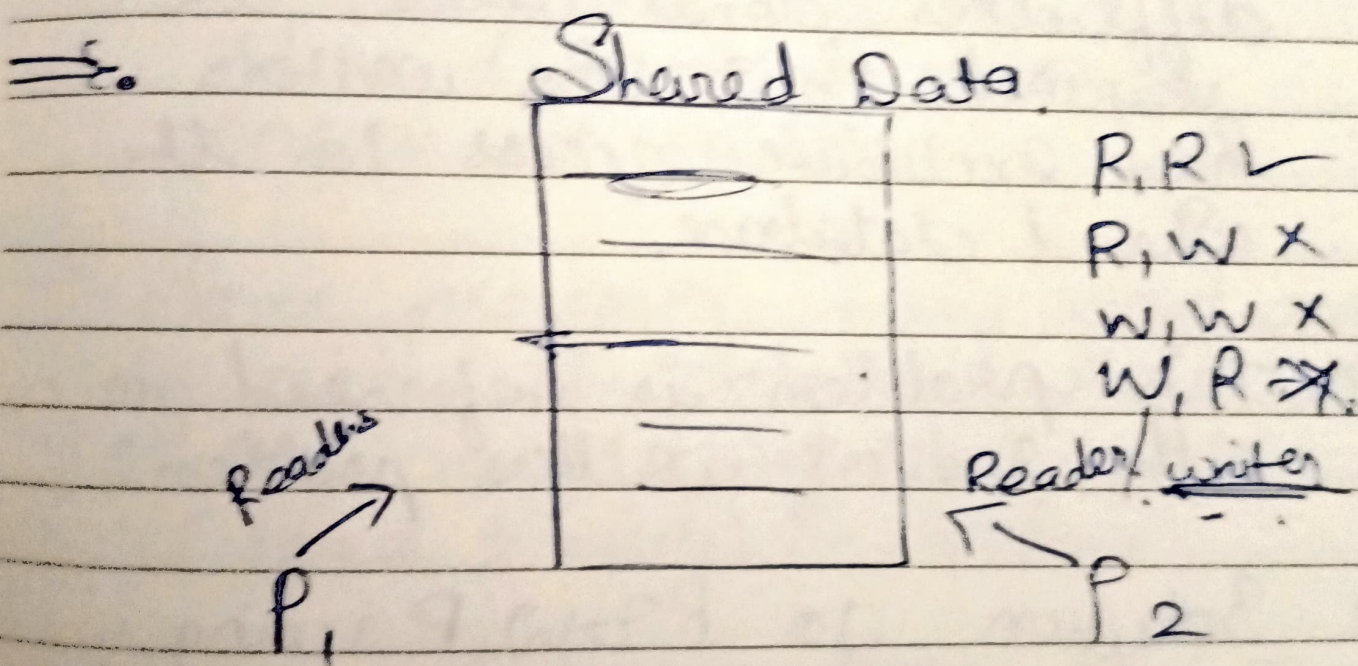
↳ 2 Semaphores & an Integer Variable.

1. **Mutex**:- it is initialized to 1.
 It is used to ensure mutual exclusion when readCount is update (when any Reader enters or exit from the CS).

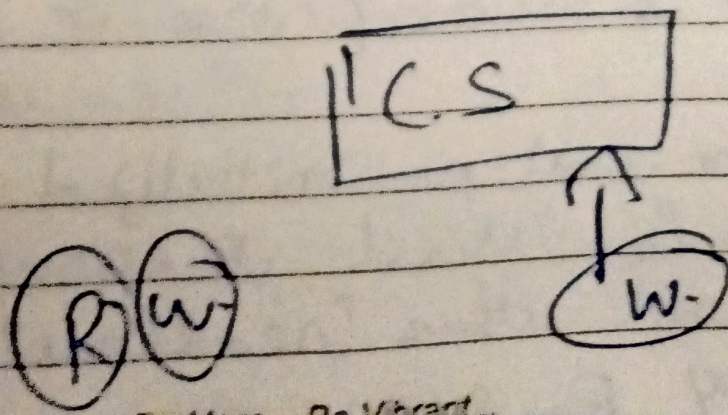
Binary.

2. Wait, right :- It is a Semaphore (initialized to 1) common to both R & W processes.

3. Readcount :- It is an integer variable (initialized to 0) that keeps track of how many processes are currently reading the data.

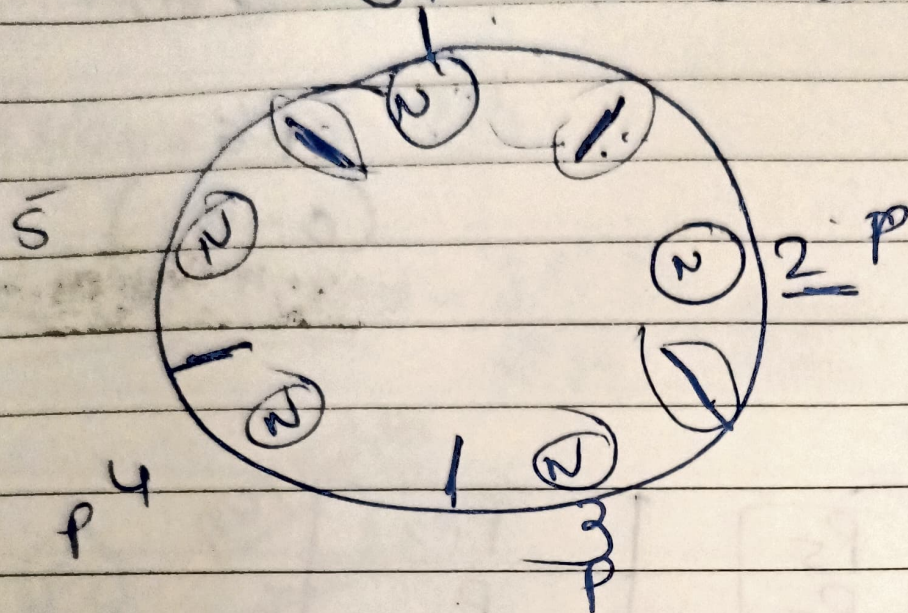


→ If Both process Readers -

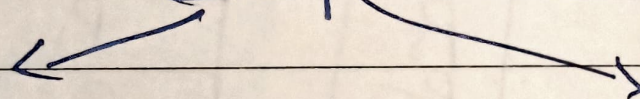


* Dining Philosophers Problem:-

$P \rightarrow$ Two State - T or E



Philosopher either



Thinks

When a philosopher thinks, he does not interact with his colleagues

Eats

When a philosopher gets hungry he tries to pick up the two forks that are closest to him (L & R). A philosopher may pick up only one fork at a time.

→ One can't pick up a fork that is